Research Question 3: Accident Severity Prediction on Long Island

AMS 597 — Statistical Machine Learning
Shane Patandin · Matthew Tranquada · Vraj Patel · Kaushal Patel · Sur Vaghasiya
Stony Brook University · Spring 2025

---

1. Research Question

Can machine learning models trained on environmental and infrastructure features predict whether a Long Island crash will be high-severity (Severity 3–4), using only information available at the time of the accident? We compare two models — Logistic Regression as an interpretable baseline and XGBoost as a gradient-boosted ensemble — and evaluate them primarily on ROC-AUC given the 5.86:1 class imbalance in the data. F1-score on the high-severity class and the Precision-Recall curve are reported alongside as supplementary diagnostics.

---

2. Data & Preprocessing

2.1 Dataset

The dataset is a cleaned subset of US Accidents filtered to Nassau, Suffolk, and Queens counties. The outcome variable `severity_binary` bins the original four-level severity scale into low (1–2, coded 0) and high (3–4, coded 1). The test set contains 6,646 records: 5,677 low-severity and 969 high-severity, giving a class ratio of roughly 5.86:1.

2.2 Feature Engineering

Twenty-six features were constructed across four domains. Two variables — `Distance(mi)` and `Duration_Minutes` — were excluded outright since both are only measurable after a crash resolves, which would introduce lookahead bias into training.

Domain	Features	Notes
Weather (continuous)	Temperature, Humidity, Pressure, Visibility, Wind Speed	Parsed to numeric; NAs imputed post-split
Temporal	hour_sin/cos, month_sin/cos, dayofweek_sin/cos, rush_hour, is_weekend, is_night	Cyclical encoding prevents hour-wrap discontinuity
Infrastructure	Junction, Traffic_Signal, Crossing, Stop	Boolean flags re-encoded as 0/1
Weather (categorical)	wx_Cloudy, wx_Rain, wx_Snow_Ice, wx_Fog_Haze, wx_Thunderstorm, wx_Other	One-hot; Clear dropped as reference level
Interactions	night×rain, night×fog, rush_hour×junction	Compound risk factors

Table 1. Feature domains used in both models.

---

3. Modelling Approach

3.1 Train/Test Split

Data was split 80/20 with stratification on the outcome to preserve the class ratio in both subsets. All preprocessing steps (scaling, imputation) live inside scikit-learn Pipelines that fit only on training data, so there is no leakage into the test set.

3.2 Logistic Regression

The LR pipeline runs StandardScaler → KNNImputer (k=5) → L1-penalised Logistic Regression (liblinear solver, max_iter=500, class_weight='balanced'). The regularisation strength C was tuned over {0.01, 0.1, 1.0, 10.0, 100.0} via 5-fold stratified cross-validation optimising ROC-AUC. L1 penalisation does double duty here — it regularises the model while also zeroing out features that don't contribute, giving a built-in form of feature selection.

3.3 XGBoost

The XGBoost pipeline applies the same preprocessing and adds a SelectFromModel step (threshold = mean gain) using a preliminary 100-tree XGBClassifier to prune features before the final model fits. The final classifier uses 300 estimators, subsample=0.8, colsample_bytree=0.8, and scale_pos_weight=5.86 to handle class imbalance. max_depth ∈ {3, 5, 7} and learning_rate ∈ {0.01, 0.05, 0.1} were jointly tuned via the same 5-fold stratified grid search.

> *Note: StandardScaler is included in the XGBoost pipeline for consistency with the LR pipeline. Tree-based models are invariant to monotone transformations, so it has no effect on the XGBoost predictions themselves.*

---

4. Results

4.1 Hold-Out Test Performance

Model	ROC-AUC	Accuracy	F1 (High Sev.)
Logistic Regression	0.6165	0.5846	0.2955
XGBoost	**0.7185**	**0.7040**	**0.3706**

Table 2. Hold-out test set metrics (n = 6,646; 969 high-severity).

XGBoost outperforms Logistic Regression across all three metrics. The 0.102 AUC gap points to non-linear structure in the data — interactions between weather, infrastructure, and time-of-day that a linear model can't fully represent even with the engineered interaction terms. Accuracy is not the headline metric here: a model that always predicted low-severity would hit ~85% accuracy while achieving an AUC of 0.50, which is no better than a coin flip on the problem that actually matters.

4.2 Classification Reports

Model	Class	Precision	Recall	F1	Support
Logistic Regression	Low (1–2)	0.89	0.58	0.71	5,677
	High (3–4)	0.20	0.60	0.30	969
	Weighted avg	0.79	0.58	0.65	6,646
XGBoost	Low (1–2)	0.91	0.72	0.81	5,677
	High (3–4)	0.27	0.60	0.37	969
	Weighted avg	0.82	0.70	0.74	6,646

Table 3. Classification reports at default threshold (0.5).

Both models achieve the same high-severity recall (0.60), but XGBoost is more precise — 0.27 vs. 0.20 — meaning fewer false positives per true high-severity catch. The shared recall value is a byproduct of the imbalance correction pushing both models toward similar sensitivity on the minority class at 0.5. The more meaningful difference between them shows up in how cleanly they classify low-severity crashes: XGBoost's low-severity recall improves from 0.58 to 0.72, which is where the AUC gap is largely coming from.

4.3 ROC Curves

Figure 1. ROC curves on the held-out test set. XGBoost AUC = 0.718; Logistic Regression AUC = 0.616.

XGBoost dominates across the full threshold range. Both models sit well above the random-classifier diagonal, confirming that the feature set carries real predictive information regardless of which model is used to extract it.

4.4 Confusion Matrices

Figure 2. Confusion matrices on the held-out test set (n = 6,646).

Logistic Regression correctly flags 579 of 969 high-severity crashes but generates 2,371 false positives among low-severity events. XGBoost catches the same 579 true positives while cutting false positives to 1,577 — a substantial reduction that explains most of the accuracy and F1 improvement. Both models share 390 false negatives (high-severity crashes called low), which is the failure mode with the most direct safety implications.

4.5 Precision-Recall Analysis

Figure 3. Precision-Recall curves. XGBoost AP = 0.327; Logistic Regression AP = 0.213; random baseline AP ≈ 0.15.

The PR curve is a more conservative read of performance under class imbalance than ROC, since it completely ignores true negatives — the large pool of correctly classified low-severity crashes that can make a mediocre model's ROC curve look better than it is. Both models clear the random baseline (AP ≈ 0.15) by a meaningful margin. XGBoost's advantage is most pronounced in the 0.0–0.4 recall range, which is the region relevant to any real deployment where you'd want a reasonably tight operating point rather than catching every possible positive.

4.6 Cross-Validation Stability

Figure 4. 5-fold cross-validation ROC-AUC with ±1 SD error bars. XGBoost: 0.7164 (±0.0042); Logistic Regression: 0.6129 (±0.0059).

CV AUCs track the test-set results closely — LR at 0.6129 vs. test 0.6165, XGBoost at 0.7164 vs. test 0.7185 — which is about as consistent as you'd hope for. The tight standard deviations (±0.004–0.006) confirm neither model is getting lucky on a particular split.

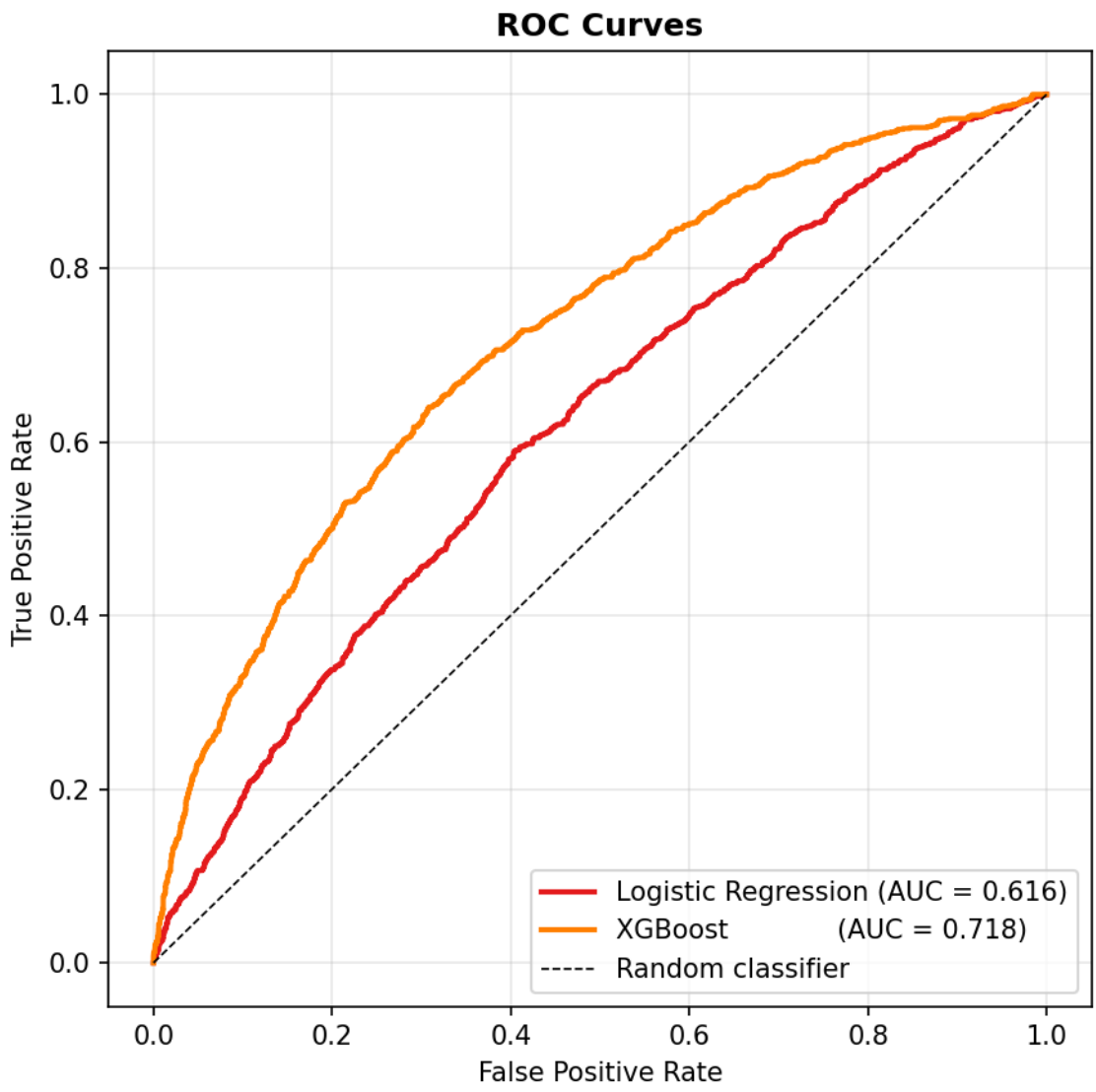

Figure 1: ROC Curves

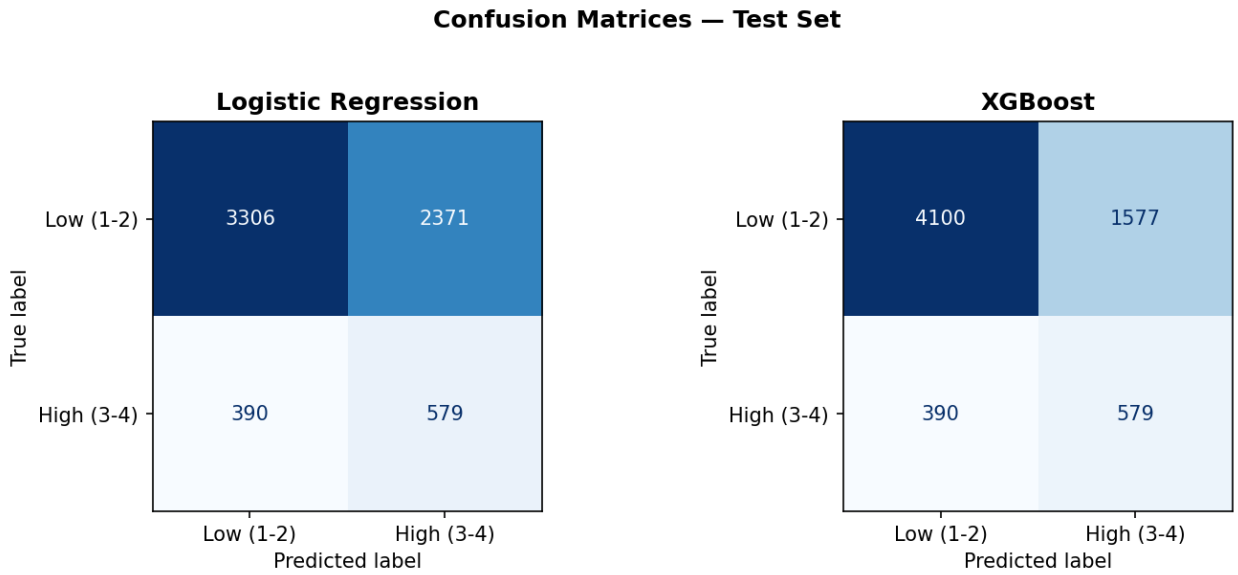

Figure 2: Confusion Matrices

4.7 XGBoost Feature Importance

Figure 5. Top-15 XGBoost features by mean gain.

Wind Speed ranks first (gain ≈ 0.085), which is somewhat unexpected given that the infrastructure features were anticipated to dominate. Crossing (≈ 0.073) and Junction (≈ 0.072) follow immediately, with the temporal encodings hour_cos, month_sin, and month_cos all clustering around 0.071. Traffic_Signal ranks seventh, and both engineered interaction terms — night_x_rain and rush_x_junction — land in the top ten, confirming they carry signal beyond their constituent features. Temperature, wx_Snow_Ice, and Humidity round out the bottom of the top 15, which is consistent with the general expectation that adverse weather conditions push crashes toward worse outcomes.

---

5. Discussion

5.1 Model Comparison

XGBoost improves on Logistic Regression by +0.102 AUC, +0.119 accuracy, +0.075 F1 on high-severity, and +0.114 Average Precision. The consistency of that gap across every metric suggests it's not noise — there is genuinely non-linear structure in how weather, infrastructure type, and time combine to determine crash severity, and the gradient-boosted ensemble is capturing it. That said, Logistic Regression isn't useless here: its coefficients are directly interpretable and can communicate directional risk to policymakers in a way that XGBoost's feature importances don't fully replicate. For prediction, XGBoost is the better tool; for explanation, LR still has a role.

5.2 Limitations

- The data is observational, so all results describe associations rather than causes. Wind Speed being the top feature doesn't mean wind causes more severe crashes — it likely correlates with broader adverse conditions that aren't fully captured in the feature set.

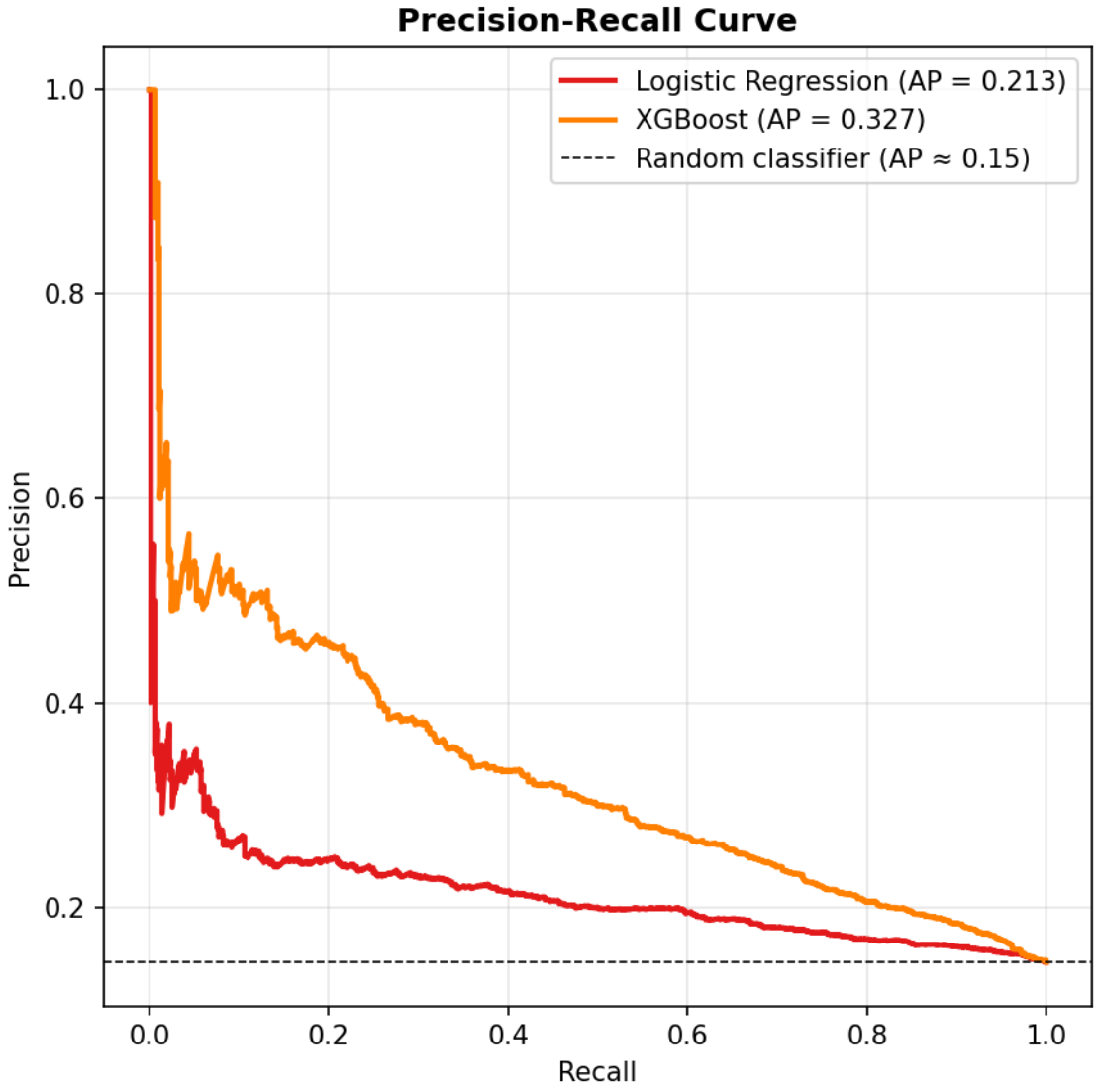

Figure 3: Precision-Recall Curve

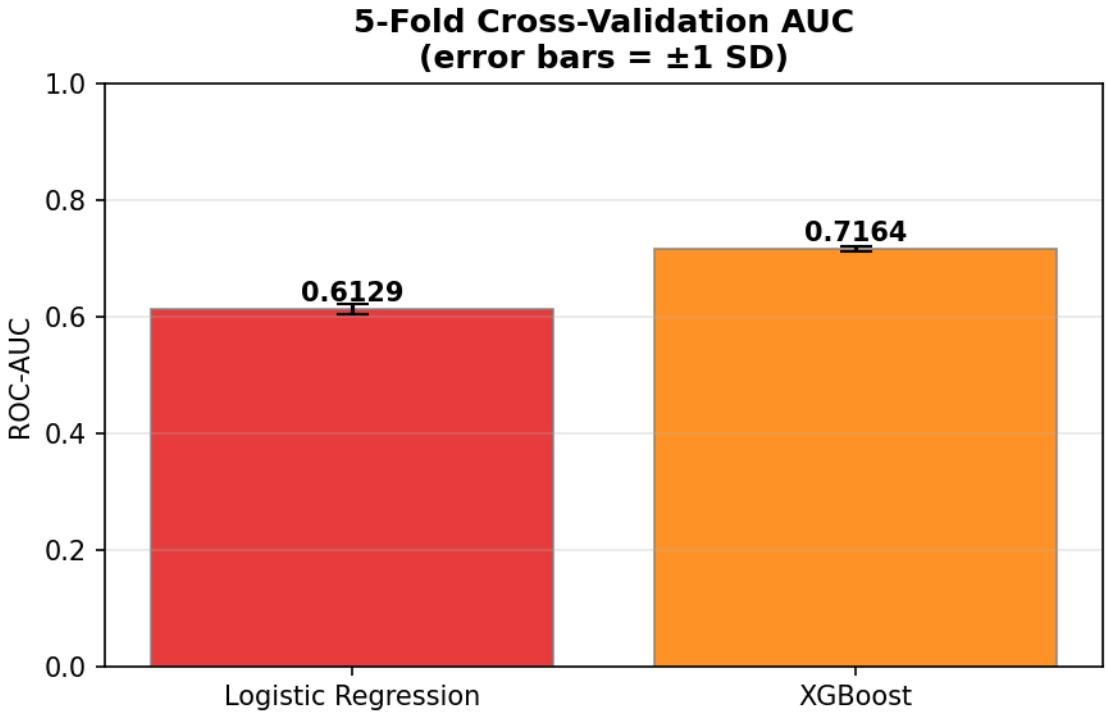

Figure 4: CV AUC Comparison

- The analysis is limited to Nassau, Suffolk, and Queens counties. Feature importances and model weights may not transfer to regions with different road networks or traffic patterns.
- Class imbalance was handled through loss re-weighting (class_weight='balanced' for LR; scale_pos_weight for XGBoost). Resampling methods like SMOTE weren't evaluated and could produce different results.
- There's no temporal holdout — the model wasn't trained on earlier crashes and tested on later ones. A time-based split would give a cleaner picture of how well the model would perform prospectively as conditions on Long Island evolve.
- The default 0.5 threshold is used throughout. Any real deployment would need an explicit operating point chosen from the PR curve depending on whether precision or recall is the priority.

---

6. Community Applications

All model inputs are observable before a crash occurs, which means the framework naturally points toward prevention rather than response. Two applications follow directly from the results.

6.1 Infrastructure Investment Prioritisation

Crossing, Junction, and Traffic_Signal rank among the top seven features by gain, and the rush_x_junction interaction term ranks ninth. This tells a fairly specific story: certain types of road infrastructure — particularly marked crossings and multi-leg junctions — are disproportionately associated with high-severity outcomes, and that association gets stronger during peak-hour congestion.

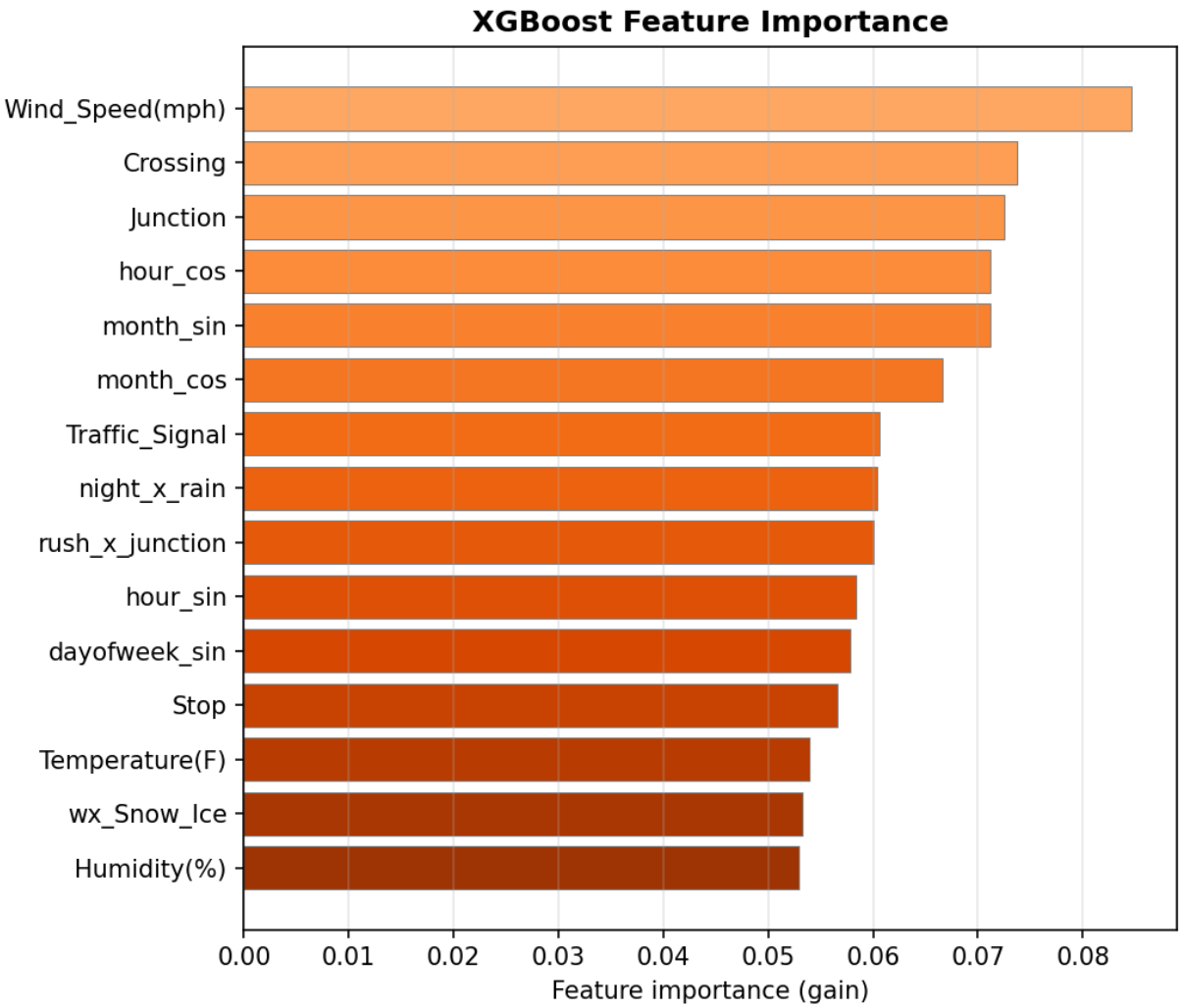

Figure 5: XGBoost Feature Importance

County transportation agencies (Nassau County DOT, Suffolk County DPW, NYC DOT for Queens) could use these rankings to build a prioritised list of intersections and crossings for engineering review. Rather than allocating safety budgets based on crash counts alone, the model offers a way to weight locations by predicted severity risk — which is a different ordering. Practical interventions at flagged sites could include signal timing adjustments during identified peak-risk windows, advanced warning signage approaching high-importance junctions, turn lane additions and pedestrian refuge islands at high-risk crossings, and retroreflective markings suited to the low-visibility weather conditions the model also flags.

6.2 Real-Time Driver Alerts

The top features — Wind Speed, hour, month, Crossing, Junction — are all available in real time from weather APIs and map data. A lightweight scoring endpoint that takes a (location, timestamp, weather observation) tuple could feed severity risk estimates to navigation apps or dynamic highway signs.

When the model sees a driver heading toward a signalised crossing during evening rush under high-wind or foggy conditions, it's looking at a combination that lands well above average predicted severity. That's a more specific trigger than a generic "fog advisory" — it accounts for the infrastructure context too. At a recall of 0.60, XGBoost's precision is around 0.27, meaning roughly one in four alerts would correspond to a genuinely high-severity situation. For a driver advisory system that's a reasonable operating point, and it can be shifted along the PR curve depending on how conservative the deployment wants to be.

---

7. Conclusion

XGBoost (AUC = 0.718, AP = 0.327) outperforms Logistic Regression (AUC = 0.616, AP = 0.213) across every metric, with the gap driven by non-linear interactions — particularly involving wind speed, infrastructure type, and time-of-day — that the linear model can't capture even with engineered interaction terms. Cross-validation results are tight and consistent with test performance, indicating the findings generalise reliably.

The feature importance analysis points toward infrastructure as the most actionable lever: Crossing, Junction, and Traffic_Signal all rank in the top seven, and the rush_x_junction interaction confirms that junction risk concentrates during peak hours. Wind Speed's rank as the single most important feature adds a weather dimension that, combined with the temporal encodings, supports both of the community applications explored here.

Infrastructure investment prioritisation and real-time driver alerting are the two use cases that follow most naturally from the model, because they work with information that's available before a crash happens rather than after. The goal in both cases is the same — fewer high-severity crashes on Long Island — and the model gives agencies and system designers a principled basis for deciding where and when to intervene.